

Project Report: Huffman Block Coding Of Books

Aleksandra Zec, Matej Kunovski, Nikola Severov

May 23, 2026

1 Introduction

This report provides an overview of an implementation of Huffman Block Encoding and its application to compression of books in different formats. The size of the Huffman block compression is compared to the sizes of standard file types. Along with this, the time of encoding was recorded, and the results were analyzed to conclude whether an approach to encoding books with Huffman was a worthwhile investment.

2 Technologies used

The project was coded in the programming language Java and versioned on GitHub. The code can be found at the following repository:

<https://github.com/aleksandrAZec/Huffman-block-coding-of-PDFs>

3 Task requirements

The project at hand requires the following steps:

1. Find a PDF file of your favorite book and read the contents of the file into a byte array.
2. Change bytes to bits to obtain a representation of the PDF file as a string of bits.
3. Split the obtained sequence of bits into blocks of 16, and find the frequency of every possible block, and use the obtained frequencies to estimate the probability of all possible blocks of 16 bits.
4. Use the estimated probability distribution to obtain a Huffman code for the blocks of 16 bits.
5. Use the Huffman code to encode the bit representation of the PDF file of the book.
6. Compare the size of the compressed file with the sizes of the same book in some other document formats (DjVu etc.).

4 Data overview

The books used for this project were found on the website Project Gutenberg [3], an online library of more than 75,000 free eBooks. Each of the books was encoded by Huffman starting from two main formats, PDF and standard Text file, using UTF-8. The following books were chosen for the encoding:

- Alice's Adventures in Wonderland by Lewis Carroll
- The Brothers Karamazov by Fyodor Dostoyevsky

These books were chosen because they have vastly different lengths, and in turn, different file sizes, bringing about diverse results.

5 Algorithm explanation

Source coding is a technique used for organizing and compressing data for the purpose of using fewer bits than the original representation, resulting in shorter transmission times and lesser storage requirements. A desirable property in source coding techniques is optimality - having the shortest average code length among any possible coding of symbols. Huffman coding is one such approach, which creates optimal encodings of symbols with a simple algorithm.

Particularly, for this project, the Huffman block coding is implemented. In essence, it is the same as the regular Huffman coding, except for the fact that instead of encoding individual symbols, several symbols are merged together in blocks, and the blocks represent the new symbols. This means that the entropy of the coding increases, and Shannon's bound on expected length of codewords is not as dramatic, as one extra bit becomes insignificant in this approach.

To explain how the algorithm works, firstly a set of all possible symbols and their probabilities (probability distribution) is required. With this, proceed as following:

1. Consider the set of all symbols along with their respective probabilities. Sort the symbols in descending order with respect to the probability distribution.
2. Merge two symbols with smallest probability, say i and j , and remove these from the set. Add a new pseudo-symbol z whose probability is $p_i + p_j$
3. If there is more than one symbol left in the set, go to the first step.
4. From the symbol of probability one - the last symbol in the set, we assign 0 to the path to one of its' parents, and 1 to the path to the other (the two symbols merged to create it). We repeat this for each of the symbols.
5. To read the codewords of the symbols, we start from the last one, and read the bits following the path to the respective symbol.

6 Implementation

The implementation of the project was done with respect to the task requirements. Firstly, the input of the algorithm is tackled. Regardless of the type of input file, its contents are read and stored into a byte array. The particular file types that were tested are: .txt, .pdf, .djvu, .epub and .mobi. Next, the data is stored into a bit array, which is used in the following steps. The bits are grouped into blocks of size 16 that are used for the algorithm. The frequencies and probabilities of the blocks are stored in HashMaps.

To explain how Huffman coding was implemented in this project, the Rosetta Code Wiki page [2] is cited:

- A Huffman encoding can be computed by first creating a tree of nodes:
 1. Create a leaf node for each symbol and add it to the priority queue.
 2. While there is more than one node in the queue:
 - (a) Remove the node of highest priority (lowest probability) twice to get two nodes.
 - (b) Create a new internal node with these two nodes as children and with probability equal to the sum of the two nodes' probabilities.
 - (c) Add the new node to the queue.
 3. The remaining node is the root node and the tree is complete.
- Traverse the constructed binary tree from root to leaves assigning and accumulating a '0' for one branch and a '1' for the other at each node. The accumulated zeros and ones at each leaf constitute a Huffman encoding for those symbols and weights.

After the encoding, the codewords for each block are gotten by following the path from root to appropriate leaf Node, and stored in a HashMap along with the respective block. Lastly, the original data is encoded block by block, where for each block, the appropriate codeword is acquired from the HashMap and appended to the resulting byte array. The result is the encoding of the original data.

7 Results and Analysis

The main file types that were taken into consideration were PDF and standard TXT file. The size of the original file and the size of the encoding were recorded in bytes, denoted by "Original Size" and "Comp. Size", respectively, as well as several timestamps in milliseconds during the process, which are:

1. Read Time - the time elapsed for reading and storing the book
2. Prob. Time - the time elapsed for calculating the probabilities of every block
3. Construct Time - the time elapsed for constructing the Huffman tree
4. Encode Time - the time elapsed for encoding the original book

5. Total Time - the total time elapsed for the whole process

The books, "Alice in Wonderland" and "The Brothers Karamazov", were encoded in both formats with the Huffman algorithm, and the following results were observed:

Book/File Type	Original Size	Comp. Size	Read Time	Prob. Time	Construct Time	Encode Time	Total Time
Alice / PDF	246901	228138	6	88	140	83	317
Alice / TXT	174309	88613	2	16	3	31	52
Karamazov / PDF	5826917	3011001	19	291	8	309	627
Karamazov / TXT	1982206	1001800	4	84	1	117	206

Table 1: Data collected for each encoded file

Observing from the table it is evident that both for "Alice in Wonderland" and "The Brother Karamazov", in both TXT and PDF, the Huffman coding compressed both books to a smaller size than the original. For the TXT files, both encodings performed really well, almost half the size of the original file.

For the PDF files, the "Alice in Wonderland" encoding did not have a significant difference to the original file, but "The Brothers Karamazov" encoding performed a lot better than the original file, almost halving the original size of the file.

The suspected reason as to why the "Alice in Wonderland" encoding of the PDF file did not have any major differences than the original, is because of the structure of a PDF. In the article, The Structure of a PDF File [1], it is explained that a PDF file has it's own structure, which contains PDF objects. They bring different combinations of bits that the blocks consist of, resulting in a larger amount of unique bit blocks that don't appear more than once or twice, and they pollute the Huffman Tree. This does not affect the "The Brothers Karamazov" encoding, because the PDF file has many pages that are just filled with text and nothing else resulting in a similar result as the TXT file.

Regarding the times needed to encode the files, it is apparent that encoding a standard TXT file takes much less time than PDF files. This, as well as the fact that the time it took to construct the Huffman tree of the "Alice in Wonderland" PDF file was abnormally high, can as well be explained by the aforementioned PDF properties. Despite the "The Brothers Karamazov" TXT file being more than 11 times larger than the "Alice in Wonderland" TXT file, the total time only took approximately four times longer. This shows an important property of the implementation - the size of the file being encoded doesn't impact the time greatly, at least as per this testing.

The time it took to calculate the probability distribution and the time it took to actually encode the book is approximately the same for all the tests, meaning that having the

probability distribution precomputed for the respective file would almost double the speed - and half the total time.

Project Gutenberg [3] provides file types other than the ones examined before. They are .epub, which are made for older E-Readers, and .mobi, which are made for Kindle devices. Additionally, the .djvu files were artificially created by us, using an online converter from .txt to .djvu.

Book	.pdf (comp.)	.txt (comp.)	.pdf	.txt	.djvu	.epub	.mobi
Alice	228138	88613	246901	174309	482974	189180	256218
Karamazov	3011001	1018597	5826917	2019466	5362983	959526	1508108

Table 2: Table of file sizes in bytes

In "Alice in Wonderland", the compressed TXT file is a 49.2% decrease from the original TXT, as well, a 64.1% decrease than the original PDF. The TXT file is an 81.7% decrease from the DjVu file, 53.2% decrease from EPUB file, and a 64.4% decrease from MOBI file. Similarly, the "The Brothers Karamazov" compressed TXT file has similar results as the "Alice in Wonderland" one. It has a 49.6% decrease from the original TXT file, and an 82.5% decrease from the original PDF file. Similarly, there is an 81% decrease from the DjVu file, a 6.2% increase from the EPUB file, and a 32.5% decrease from the MOBI file.

The observed compressed TXT file has a significant decrease with all other file types, except the one case, where "The Brothers Karamazov" compressed TXT has an increase than their EPUB file. This might be due to the EPUB file being smaller than the original TXT, which is not a usual occurrence in other books (observed with "Alice in Wonderland").

8 Conclusion

Based on the analysis of the recorded results, it can be concluded that the Huffman block coding is worthwhile for the compression of books, since in less than a second, it can compress the book into less than half of its original file size, if done with the proper care. Compressing of file types that are more complex than a standard TXT file might not deliver the same results. Precomputing the probability distribution, not for a specific file, but for the English language as a whole, might result in a standardized and re-applicable approach that can be used for the compression of many books, despite possibly not delivering as efficient of a compression as calculating the probability distribution individually.

Further expansion of this project would entail changing of the block sizes, and testing which would result in better compression, while still not taking significantly more time. The implementation of this project is best applicable in situations where saving storage is of utmost importance, and the time might not matter that much. This could be useful for digital archiving of books and other forms of text files.

References

- [1] Jay Berkenbilt. *The Structure of a PDF File*. <https://medium.com/@jberkenbilt/the-structure-of-a-pdf-file-6f08114a58f6>.
- [2] Rosetta Code. *Huffman coding*. https://rosettacode.org/wiki/Huffman_coding. 2025.
- [3] Michael Hart. *Project Gutenberg*. <https://www.gutenberg.org/>.
- [4] Sandeep Jain. *GeeksForGeeks*. <https://www.geeksforgeeks.org/dsa/huffman-coding-greedy-algo-3/>.
- [5] Jon Skeet. *Stack Overflow*. <https://stackoverflow.com/questions/1131116/pdf-to-byte-array-and-vice-versa>.